LAB 20

Zero-Downtime Deployment

Koordinasi Probe, PreStop Hook, dan Database Parallel Change di Go

#Zero-Downtime #PreStop-Hook #Expand-Contract #Cooperative-Worker #Go-Stdlib

server.go (graceful preStop shutdown)

```
// 1. Detach from load balancer s.SetReady(false) // 2. PreStop
delay for routing propagation select { case <-
time.After(s.preStop): case <-ctx.Done(): return ctx.Err() } // 3.
Graceful HTTP connection draining err := s.srv.Shutdown(ctx)
s.wg.Wait() >> PASS: 0 in-flight requests dropped
```

AUTHOR: lukman-ss / Lukman

Masalah: Penghentian Instan Memutus Trafik

Menghentikan instance aplikasi tanpa koordinasi memicu kegagalan sistem.

MOTIVASI

Setiap kali versi baru dirilis, mematikan instance lama secara mendadak memutus ribuan request HTTP yang sedang berjalan (in-flight) di tengah jalan. Hal ini memicu lonjakan error HTTP 502 Bad Gateway dan koneksi terputus ke pengguna.

BATAS LAB

Lab ini membuktikan arsitektur zero-downtime di Go: pemisahan liveness/readiness probe, graceful HTTP draining, preStop hook delay, terminasi worker kooperatif, dan fallback skema database Expand/Contract.

Never terminate old instances before new instances are ready to serve.

Mental Model: Koeksistensi Versi N & N-1

Versi baru dan versi lama wajib hidup berdampingan saat proses rollout.

PRINSIP UTAMA

Zero-downtime deployment bukanlah peralihan seketika (instant flip). Versi N (baru) dan Versi N-1 (lama) berjalan bersamaan di infrastruktur yang sama, mengakses basis data yang sama, dan melayani trafik secara paralel.

TIGA PILAR KOEKSISTENSI

1. Perutean Trafik: Load balancer hanya mengirim trafik ke pod yang berstatus Ready.
2. Connection Draining: Pod lama menyelesaikan tugas in-flight.
3. Skema Kompatibel: Database melayani pembacaan dan penulisan dari kedua versi secara simultan.

Start new before stopping old: coexistence is the foundation of zero downtime.

Pemisahan Liveness dan Readiness Probe

Dua mekanisme pemeriksaan kesehatan dengan tanggung jawab yang berbeda.

LIVENESS PROBE (/healthz/live)

Mendeteksi apakah proses aplikasi masih hidup dan tidak hang atau deadlock. Jika liveness gagal (non-200), orkestrator seperti Kubernetes langsung me-restart kontainer. Probe ini harus tetap 200 selama proses shutdown tertib.

READINESS PROBE (/healthz/ready)

Mendeteksi apakah instance siap menerima paket lalu lintas dari load balancer. Begitu instance menerima sinyal shutdown, status readiness langsung diubah menjadi 503 untuk melepas pod dari rotasi jaringan.

Liveness determines if the process lives; Readiness determines if it takes traffic.

Celah Propagasi Routing Asinkron

Sinyal SIGTERM tiba sebelum aturan jaringan load balancer selesai diperbarui.

DUA JALUR ASINKRON

Di Kubernetes, sinyal SIGTERM ke pod (control plane) dan pembaruan EndpointSlice ke kube-proxy/ingress (data plane) berjalan secara asinkron dan paralel. Terdapat jeda beberapa ratus milidetik hingga beberapa detik.

AKIBAT FATAL

Jika server HTTP langsung mematikan listener seketika saat menerima SIGTERM, paket trafik baru yang masih dikirim oleh load balancer akan menemukan port tertutup, menghasilkan HTTP 502 Bad Gateway kepada pengguna.

Pod termination and network endpoint detachment are asynchronous in Kubernetes.

Solusi: PreStop Delay Menjembatani Routing

Memberikan jeda waktu agar tabel routing eksternal sempat menyinkronkan status.

MEKANISME PRESTOP

Saat menerima sinyal shutdown, server segera menandai dirinya unready pada readiness probe, lalu menahan proses selama durasi preStop (misal 1-5 detik) sebelum memanggil shutdown listener HTTP.

HASIL OPERASIONAL

Selama masa penundaan, server tetap melayani request in-flight dan sisa paket yang terlanjur terkirim. Begitu tabel routing jaringan selesai diperbarui, aliran request baru berhenti secara bersih.

A preStop delay gives network ingress controllers time to detach the pod.

Implementasi PreStop Berbasis Select Timer

Menghindari sleep telanjang agar penundaan tetap dapat dibatalkan konteks.

KODE SERVER.GO

```
if s.preStop > 0 {
select {
case <-time.After(s.preStop):
case <-ctx.Done():
return ctx.Err()
}
}
srv.Shutdown(ctx)
```

KEAMANAN KONTEKS

Jangan pernah menggunakan time.Sleep biasa! Menggunakan select dengan ctx.Done() memastikan jika orkestrator membatalkan context timeout, penundaan berhenti seketika tanpa mengunci proses sebagai zombie.

Always wrap delay timers inside select with context cancellation support.

Connection Draining pada HTTP Server

Menutup penerimaan koneksi baru dan membiarkan koneksi in-flight selesai.

METODE SRV.SHUTDOWN

Paket standar net/http menyediakan method srv.Shutdown(ctx). Fungsi ini menutup listener port agar tidak ada koneksi TCP baru yang diterima, lalu menunggu seluruh koneksi aktif selesai merespons.

PELACAKAN GOROUTINE

Handler /work melacak goroutine aktif menggunakan sync.WaitGroup (s.wg.Add(1) dan defer s.wg.Done()). Pada akhir method Shutdown, s.wg.Wait() menjamin semua operasi tuntas sebelum proses keluar.

Connection draining allows in-flight HTTP requests to complete before process exit.

Uji Otomatis: Server Draining & PreStop

Verifikasi deterministik terhadap penyelesaian request dan kepatuhan durasi jeda.

TEST SERVER DRAINING

TestServerGracefulShutdown mengirim request /work berdurasi 150ms, lalu memicu Shutdown di tengah eksekusi. Hasil: request tuntas dengan status 200 OK dan server berhenti bersih tanpa error.

TEST PRESTOP HOOK

TestServerPreStopHook memverifikasi bahwa shutdown membutuhkan waktu tepat >=100ms sesuai konfigurasi s.preStop. Test cancellation membuktikan jeda terpotong instan saat context dibatalkan.

Automated tests prove zero dropped requests during server shutdown.

Tantangan Terbesar: Skema Database

Mengubah skema database secara langsung memicu error fatal pada versi lama.

MASALAH SCHEMA DRIFT

Aplikasi dapat di-deploy bertahap (canary atau rolling update), namun database bersifat tunggal (stateful shared resource). Mengubah skema secara destruktif memutus kompatibilitas aplikasi yang sedang berjalan.

CONTOH KASUS FATAL

Jika kolom 'name' diganti nama menjadi 'first_name' dan 'last_name', pod versi lama (v1) yang masih menerima trafik akan langsung crash dengan SQL error: column 'name' does not exist.

Database schema changes must be strictly backward compatible during deployment.

Pola Expand and Contract (Parallel Change)

Membagi evolusi skema ke dalam tahapan independen tanpa memutus sistem.

FASE 1: EXPAND

Tambahkan kolom baru ('first_name', 'last_name') sebagai kolom nullable. Kolom lama ('name') tetap dipertahankan. Deploy aplikasi versi N yang mendukung dual-write dan fallback-read.

FASE 2: MIGRATE & CONTRACT

Migrate: Jalankan script background untuk backfill data lama ke kolom baru.
Contract: Setelah seluruh pod versi lama dipensiunkan, kolom usang ('name') baru dihapus pada rilis terpisah.

Never alter or drop a database column in the same deployment that introduces it.

Implementasi Fallback Reading di Go

Menangani pembacaan data legacy dan modern secara transparan di lapisan data.

LOGIKA GETUSER (DB.GO)

```
if rec.FirstName == "" && rec.LastName == "" &&
rec.Name != "" {
parts := strings.SplitN(rec.Name, " ", 2)
rec.FirstName = parts[0]
rec.LastName = parts[1]
} else if rec.Name == "" {
rec.Name = rec.FirstName + " " + rec.LastName
}
```

DUAL COMPATIBILITY

Jika data disimpan oleh aplikasi versi lama, aplikasi baru memecah string Name on-the-fly. Jika data disimpan oleh aplikasi baru, string Name disintesis secara transparan. Tidak ada kegagalan query.

Transparent fallback logic allows modern and legacy records to coexist safely.

Uji Otomatis: Kompatibilitas Database

Memvalidasi pembacaan dan penimpaan record lintas generasi skema.

TEST EXPAND CONTRACT

TestExpandContractDatabase menguji penyimpanan record warisan via InsertLegacy dan record modern via SaveExpand. Kedua record berhasil dibaca ulang dengan field lengkap tanpa kehilangan data.

TEST OVERWRITE RECORD

TestDBLegacyOverwriteWithExpand membuktikan record yang awalnya ditulis dalam format warisan dapat ditimpa oleh skema modern dan tetap terbaca utuh pada kedua format.

Database schema tests verify zero data corruption across legacy and modern formats.

Bahaya DDL Table Lock di Produksi

Kunci ACCESS EXCLUSIVE dapat membekukan kueri aplikasi jika tidak dibatasi.

CONSTANT VS VOLATILE DEFAULT

Di PostgreSQL 11+, ADD COLUMN dengan default konstan (DEFAULT 'active') adalah operasi metadata instan tanpa rewrite. Namun default volatil (clock_timestamp(), uuid) memaksa full table rewrite!

MITIGASI LOCK_TIMEOUT

Setiap perintah DDL meminta ACCESS EXCLUSIVE lock. Jika antrean terhambat kueri lambat, kueri lain akan menumpuk. Di produksi, selalu tetapkan: SET lock_timeout = '2s'; sebelum menjalankan migrasi DDL.

Always guard production schema migrations with explicit lock_timeout parameters.

Penghentian Background Queue Worker

Menghentikan pemrosesan asinkron tanpa merusak integritas job yang sedang aktif.

PERBEDAAN DENGAN HTTP

Server HTTP menangani request berdurasi milidetik. Background queue worker memproses pekerjaan berdurasi panjang (detik hingga menit). Menghentikan worker secara paksa memicu corrupt state atau eksekusi ganda.

KONTRAK PENGHENTIAN

Worker kooperatif merespons sinyal shutdown dengan: 1) Berhenti mengambil job baru dari queue, 2) Menuntaskan pekerjaan yang sedang aktif, dan 3) Menghormati batas waktu pengurasan (drain timeout).

Queue workers must finish active jobs before gracefully yielding execution.

Implementasi Loop Worker Kooperatif

Pemeriksaan ganda konteks pembatalan sebelum dan selama pemrosesan tugas.

KODE WORKER.GO

```
select {
case <-w.ctx.Done(): return
case job, ok := <-w.jobChan:
if !ok { return }
select {
case <-time.After(job.Duration):
w.completed = append(w.completed, job.ID)
case <-w.ctx.Done(): return
}
}
```

PENCEGAHAN DEADLOCK

Pekerja mendengarkan w.ctx.Done() pada setiap tahap: sebelum dequeue, setelah dequeue, dan saat mengeksekusi durasi tugas. Jika drain timeout tercapai, konteks dibatalkan dan goroutine keluar tertib.

Cooperative workers check context cancellation at every execution boundary.

Uji Otomatis: Worker Termination

Pembuktian penyelesaian tugas aktif dan penolakan tugas baru pasca-stop.

TEST GRACEFUL STOP

TestWorkerGracefulShutdown memasukkan 2 job. Saat job 1 berjalan, sinyal stop dikirim. Worker menyelesaikan job 1 dan job 2 yang tersisa di channel, lalu berhenti tanpa kehilangan tugas.

TEST DRAIN TIMEOUT

TestWorkerShutdownTimeout menguji job lambat yang melebihi drain timeout. Worker membatalkan konteks secara bersih dan keluar tanpa blocking selamanya.

Queue tests prove active jobs finish cleanly while post-shutdown jobs are rejected.

Orkestrasi Demo: Simulasi Siklus Penuh

Menggabungkan server, worker, database, dan sinyal OS dalam satu skenario nyata.

ALUR CMD/DEMO

1. Inisialisasi Database (Expand/Contract)
2. Mulai HTTP Server (preStop 1s) & Worker
3. Kirim in-flight request HTTP (/work 300ms)
4. Simulasikan SIGTERM dari orkestrator
5. Eksekusi preStop delay, server drain, & worker stop

VERIFIKASI OUTPUT

Eksekusi go run ./cmd/demo membuktikan:
- Request HTTP tuntas 200 OK selama shutdown
- Worker menyelesaikan tugas aktif
- Server dan worker berhenti dengan status exit 0 tanpa kebocoran interaksi.

The CLI demo orchestrates all components under simulated SIGTERM conditions.

Matriks Verifikasi Pengujian Otomatis

Seluruh 16 kasus uji lulus secara deterministik dengan nol race condition.

HASIL EKSEKUSI SUITE

- TestExpandContractDatabase: PASS
- TestDBLegacyOverwriteWithExpand: PASS
- TestServerProbes: PASS
- TestServerGracefulShutdown: PASS
- TestServerPreStopHook: PASS
- TestServerPreStopCancellation: PASS

HASIL WORKER & CONCURRENCY

- TestServerMultiRequestDrain: PASS
- TestWorkerConcurrency: PASS
- TestWorkerGracefulShutdown: PASS
- TestWorkerShutdownTimeout: PASS
- TestWorkerConcurrentEnqueueStop: PASS
- go test -race: 0 data races (PASS)

16/16 automated test assertions pass cleanly under race detector verification.

Hierarki Timeout pada Orkestrator

Menyelaraskan durasi timeout untuk mencegah pembunuhan paksa oleh SIGKILL.

URUTAN TIMEOUT WAJIB

preStop delay (1-5s)
< HTTP Draining Timeout (10-15s)
< terminationGracePeriodSeconds (30-60s)

BAHAYA MISKONFIGURASI

Jika terminationGracePeriodSeconds lebih pendek daripada total preStop + HTTP draining, Kubernetes akan mengirim SIGKILL sebelum request tuntas, merusak seluruh manfaat graceful shutdown.

Termination grace period must exceed the sum of preStop delay and drain timeouts.

HTTP Worker vs Background Queue Worker

Karakteristik terminasi sinyal pada runtime aplikasi web dan queue processor.

HTTP WORKER (PHP-FPM / GO)

PHP-FPM tidak menangani SIGTERM secara bawaan untuk menyelesaikan in-flight HTTP request. Butuh konfigurasi process_control_timeout di php-fpm.conf dan proxy_read_timeout di nginx untuk connection draining.

QUEUE WORKER (SUPERVISOR / HORIZON)

Laravel Horizon menyediakan command horizon:terminate yang menunggu worker menyelesaikan job aktif. Supervisor memerlukan konfigurasi stopwaitsecs yang cukup panjang agar tidak mengirim SIGKILL prematur.

Tune process managers and proxies specifically for HTTP versus queue workloads.

Anti-Patterns & Kesalahan Umum

Jebakan implementasi yang sering membatalkan janji zero-downtime.

JEBAKAN OPERASIONAL

1. Menggabungkan Liveness & Readiness: Pod di-restart saat sedang sibuk inisialisasi.
2. Melewatkan PreStop Hook: Menghasilkan lonjakan HTTP 502 akibat propagasi rute lambat.
3. time.Sleep Tanpa Konteks: Proses menggantung dan tidak bisa dibatalkan.

JEBAKAN DATABASE & QUEUE

4. Rename Kolom dalam 1 Rilis: Menghancurkan pod versi lama seketika.
5. Volatile Default DDL: Memicu table lock eksklusif berjam-jam.
6. Worker SIGKILL Instan: Merusak job status atau memicu eksekusi duplikat.

Eliminate deployment anti-patterns to guarantee truly unnoticeable releases.

Checklist Produksi Zero-Downtime

Standar verifikasi sebelum mendistribusikan pembaruan ke lingkungan produksi.

INFRASTRUKTUR & APLIKASI

[ ] Pisahkan endpoint /healthz/live dan /healthz/ready
[ ] Konfigurasi preStop hook delay (1-5 detik)
[ ] Implementasikan srv.Shutdown dengan WaitGroup tracking
[ ] Pastikan terminationGracePeriodSeconds mencukupi total jeda

DATABASE & WORKER

[ ] Terapkan pola Expand and Contract untuk setiap mutasi skema
[ ] Lindungi migrasi DDL dengan SET lock_timeout = '2s'
[ ] Gunakan cooperative termination pada background worker
[ ] Konfigurasi stopwaitsecs supervisor worker sesuai durasi job

Verify probes, network delays, database phases, and worker timeouts before release.

SOFTWARE ENGINEERING LAB | LAB 20

Start New Before Stopping Old: Coexistence, Probes, & Parallel Change.

Zero-downtime deployment relies on version coexistence, decoupled health probes, preStop network delays, HTTP connection draining, and backward-compatible database schema evolution.

• Verifikasi Otomatis: 16/16 PASS (go test -race ./...) • Kunci Arsitektur: PreStop select-timer, net/http draining, Expand/Contract DB, Cooperative worker • Repositori: github.com/lukman-ss/software-engineering-lab/tree/main/labs/20-zero-downtime-deployment • Komitmen Terverifikasi: 0489062

Karya: Lukman / lukman-ss • Software Engineering Lab Seri 20